ADESH PATANKAR

BDT LAB: 09

Problem Statement: Implement the installation of Apache Spark

Outcome: Students will be able to install Apache Spark on their systems.

Theory: Apache Spark is a lightning-fast, open-source data-processing engine for machine learning and AI applications, backed by the largest open-source community in big data. Apache Spark (Spark) easily handles large-scale data sets and is a fast, general-purpose clustering system that is well-suited for PySpark. It is designed to deliver the computational speed, scalability, and programmability required for big data—specifically for streaming data, graph data, analytics, machine learning, large-scale data processing, and artificial intelligence (AI) applications.

Spark's analytics engine processes data 10 to 100 times faster than some alternatives, such as Hadoop (link resides outside ibm.com) for smaller workloads. It scales by distributing processing workflows across large clusters of computers, with built-in parallelism and fault tolerance. It even includes APIs for programming languages that are popular among data analysts and data scientists, including Scala, Java, Python, and R.

Spark is often compared to Apache Hadoop, and specifically to Hadoop MapReduce, Hadoop's native data-processing component. The chief difference between Spark and MapReduce is that Spark processes and keeps the data in memory for subsequent steps—without writing to or reading from disk—which results in dramatically faster processing speeds.

Spark was developed in 2009 at UC Berkeley's AMPLab. Today, it is maintained by the Apache Software Foundation and boasts the largest open-source community in big data, with over 1,000 contributors. It's also included as a core component of several commercial big data offerings.

Features of Apache Spark: Apache Spark is a powerful open-source distributed computing system that offers a wide range of features for processing large-scale data sets.

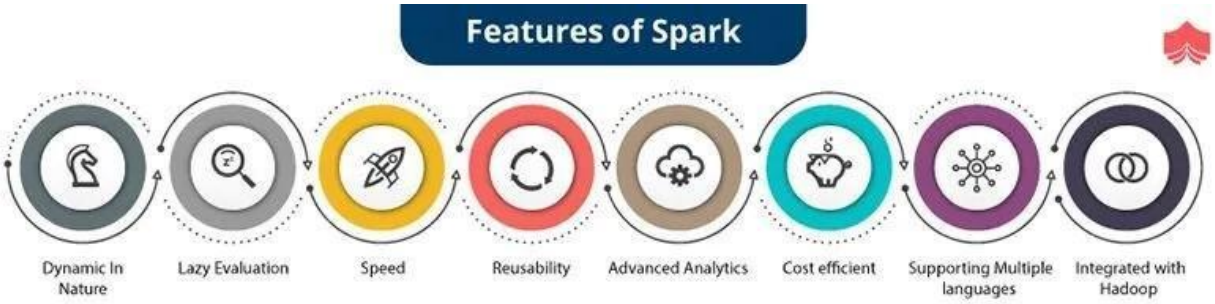


Fig. 1. Features of Spark

Here are some of its key features:

In-Memory Computing: Spark's ability to cache data in memory allows for faster data processing compared to disk-based systems.

Fault Tolerance: Spark provides fault tolerance through resilient distributed datasets (RDDs), allowing it to recover from failures gracefully.

Versatility: Spark supports various programming languages, including Scala, Java, Python, and R, making it accessible to a wide range of developers.

Unified Analytics Engine: Spark integrates seamlessly with other big data tools and libraries, such as Hadoop, Hive, and MLlib, enabling end-to-end data analytics workflows.

Community and Ecosystem: Spark has a vibrant open-source community and a rich ecosystem of libraries and tools that extend its functionality. These libraries include Spark Streaming, Spark SQL, MLlib, GraphX, and more. Additionally, Spark integrates seamlessly with other big data tools and frameworks, such as Hadoop, Hive, Kafka, and TensorFlow.

Scalability and Performance: Spark is designed to scale horizontally, allowing it to handle large-scale data processing tasks efficiently. It can run on clusters with thousands of nodes and petabytes of data. Moreover, Spark's performance optimizations, such as in-memory caching and query optimization, contribute to its high performance.

Ease of Use: Spark provides high-level APIs and abstractions that simplify the development of distributed data processing applications. Developers can write concise and expressive code using Spark's APIs, which abstract away the complexities of distributed computing.

Apache Spark is a versatile and powerful framework that enables developers to build scalable and high-performance data processing applications for a wide range of use cases, from batch processing to real-time analytics and machine learning.

Architecture of Apache Spark: Apache Spark has a well-defined layered architecture where all the spark components and layers are loosely coupled. This architecture is further integrated with various extensions and libraries. Apache Spark Architecture is based on two main abstractions:

Resilient Distributed Dataset (RDD)

Directed Acyclic Graph (DAG)

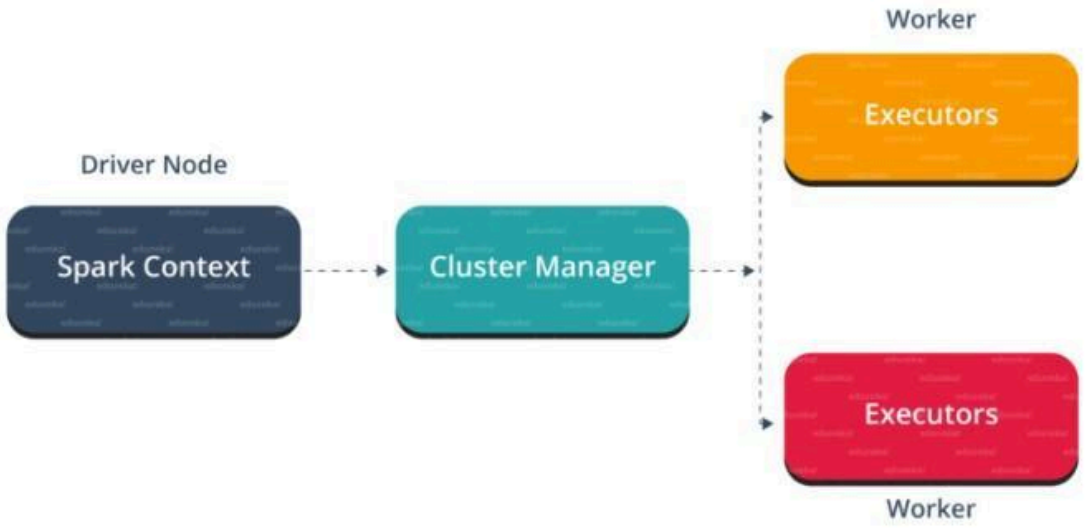


Fig. 2. Spark Architecture

Spark Eco-System: The Spark ecosystem comprises various components and libraries that extend its functionality for different data processing tasks:



Fig. 3. Spark Eco-System

Spark Core: The foundational component for large-scale parallel and distributed data processing. It provides essential functionalities for memory management, fault recovery, job scheduling, and interaction with storage systems.

Spark SQL: Integrates relational processing with Spark's functional programming API. It supports querying data using SQL or the Hive Query Language, making it easier for users familiar with relational databases to work with Spark.

Spark Streaming: Processes real-time streaming data, enabling high-throughput and fault-tolerant stream processing of live data streams. It seamlessly integrates with Spark Core, allowing users to combine batch and streaming data processing.

GraphX: Provides an API for graphs and graph-parallel computation. It extends the Spark RDD with a Resilient Distributed Property Graph, enabling users to perform graph processing and analysis tasks efficiently.

MLlib (Machine Learning): A scalable machine learning library for Apache Spark. It offers a wide range of machine learning algorithms and utilities for building and deploying machine learning models at scale.

SparkR: An R package that provides a distributed data frame implementation. It allows R users to perform data manipulation and analysis on large datasets using Spark's distributed computing capabilities.

Resilient Distributed Dataset (RDD)

RDD is the fundamental data structure in Spark, serving as the building block for Spark applications. Key characteristics of RDDs include:

Resilient: RDDs are fault-tolerant and capable of recovering data on failure. They achieve fault tolerance by replicating data across multiple nodes in the cluster.

Distributed: RDDs distribute data across multiple nodes in a cluster, enabling parallel processing of data.

Dataset: RDDs represent a distributed collection of partitioned data with values. They are immutable and support lazy transformations, meaning operations on RDDs are evaluated only when an action is triggered.

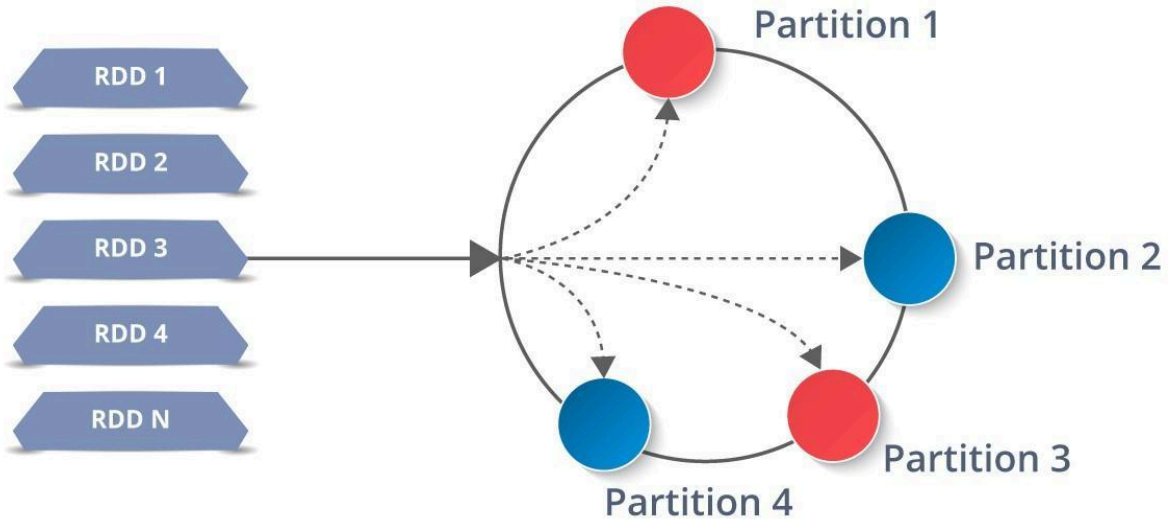


Fig. 4. RDD

RDDs can be created from external data sources such as HDFS or by parallelizing an existing collection in the driver program. Transformations and actions are the two types of operations performed on RDDs, where transformations create new RDDs, and actions instruct Spark to perform computation and return results to the driver.

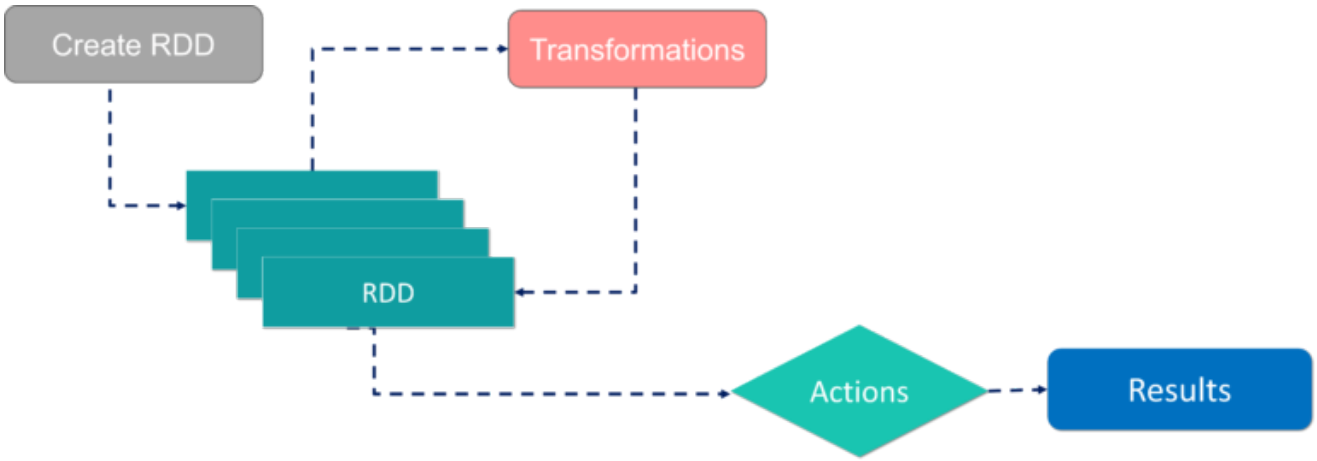


Fig. 5. Workflow of RDDs

RDDs form the backbone of Spark's distributed computing model, allowing users to perform scalable and fault-tolerant data processing across distributed computing environments.

Working of Apache Spark:

Apache Spark, a distributed computing system, processes vast datasets across clusters of computers. Its working involves several key steps:

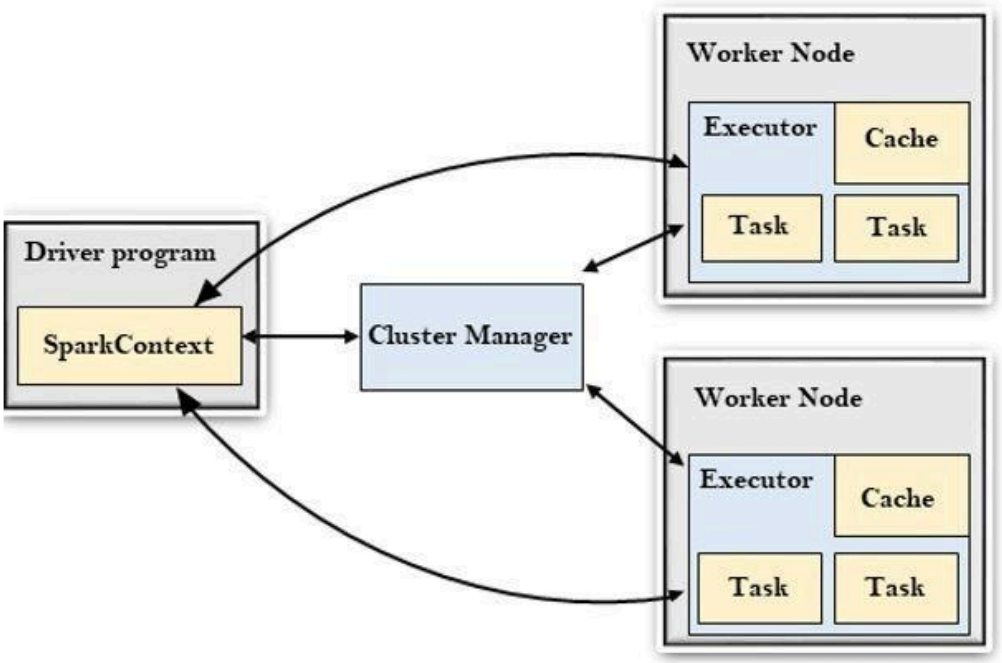


Fig. 6. Working of Apache spark

1. **Cluster Setup:**

Spark applications run on a cluster of machines managed by a cluster manager like Apache YARN, Apache Mesos, or Spark's standalone mode.

Each cluster contains one master node and multiple worker nodes. Worker nodes execute tasks and store data, while the master node manages the overall execution.

2. **Data Ingestion:**

Data is ingested from various sources like Hadoop Distributed File System (HDFS), Amazon S3, Apache Cassandra, relational databases, etc.

Spark can read data in different formats such as CSV, JSON, Parquet, Avro, and more.

3. **Resilient Distributed Datasets (RDDs):**

RDDs are the fundamental data abstraction in Spark.

They represent distributed collections of immutable data partitions that can be processed in parallel across the cluster.

RDDs are created either by parallelizing an existing collection in the driver program or by referencing a dataset in an external storage system.

4. **Transformation and Action Operations:**

Spark offers two types of operations on RDDs: transformations and actions.

Transformations create new RDDs from existing ones by applying functions to each element of the dataset. Examples include `map`, `filter`, `reduceByKey`, etc.

Actions perform computations on RDDs and return results to the driver program or write output to external storage. Examples include `collect`, `count`, `saveAsTextFile`, etc.

5. **Lazy Evaluation and DAG Execution:**

Transformations are lazy, meaning they are not executed immediately but instead build up a Directed Acyclic Graph (DAG) of operations.

When an action is called, Spark evaluates the DAG and executes the necessary transformations to compute the result.

6. **Task Execution:**

Spark's execution engine breaks down the computation into smaller tasks and schedules them to run on worker nodes in the cluster.

Tasks are executed in parallel across the cluster, with each task processing one partition of the RDD.

Executor processes running on worker nodes manage task execution and communicate with the driver to fetch data, execute computations, and return results.

7. **Fault Tolerance:**

Spark provides fault tolerance through RDD lineage, allowing lost data partitions to be recomputed using the original dataset or intermediate transformations.

RDDs are immutable and can be rebuilt from the source data or derived RDDs.

8. **Optimization:**

Spark's execution engine applies various optimizations to improve performance, such as pipelining transformations, partitioning data for parallelism, and caching intermediate results in memory.

Catalyst optimizer optimizes SQL queries and DataFrame operations to generate efficient execution plans.

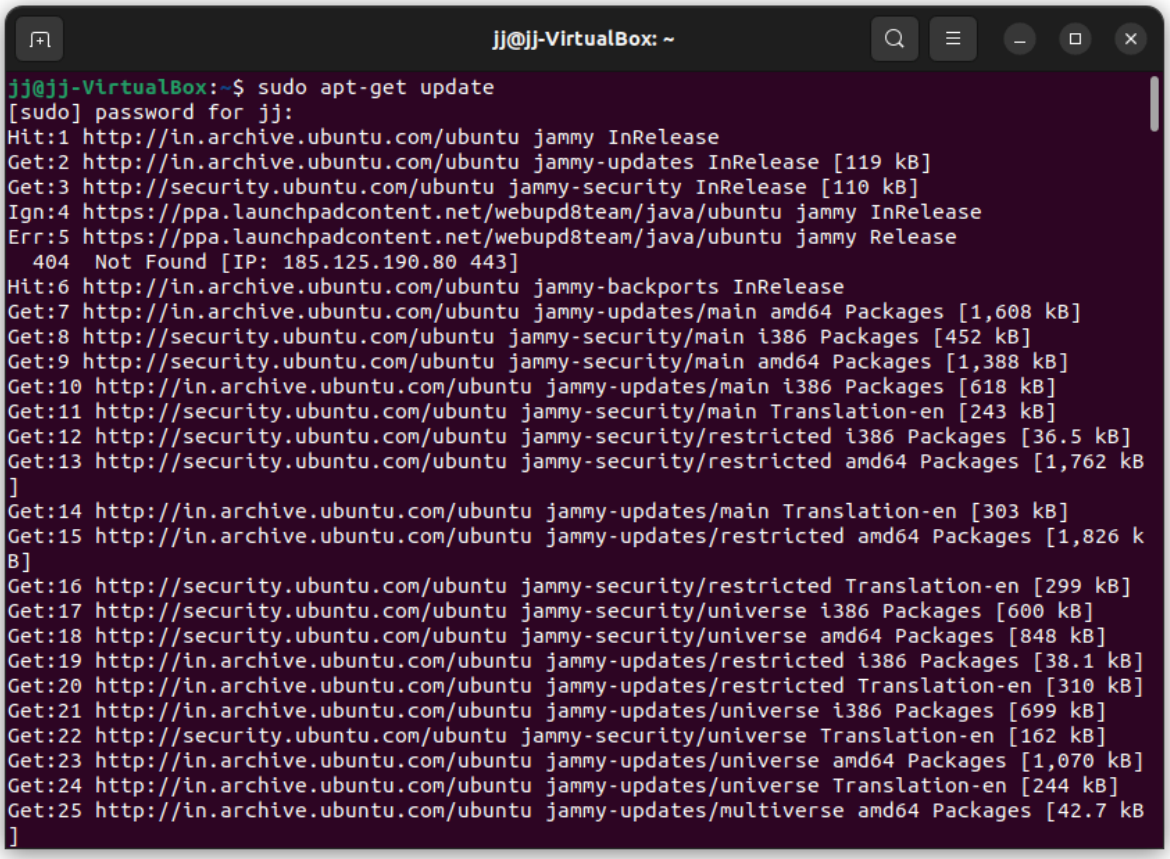
9. **Resource Management:**

Resource managers like YARN or Mesos allocate resources to Spark applications based on their requirements and monitor their execution to ensure efficient resource utilization.

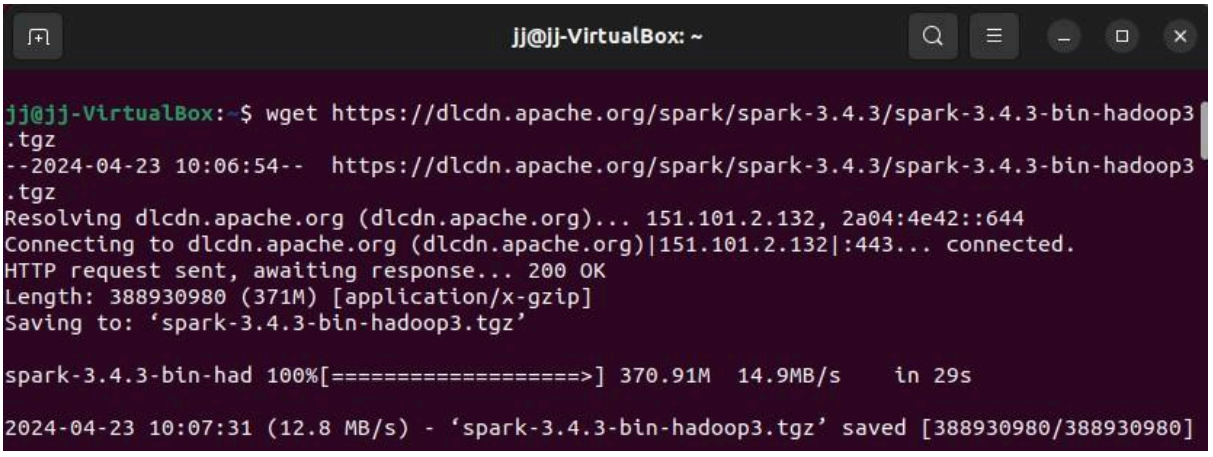
Overall, Apache Spark's distributed computing model enables scalable, fault-tolerant, and high-performance data processing across large clusters of machines, making it suitable for a wide range of big data analytics and machine learning tasks.

Results:

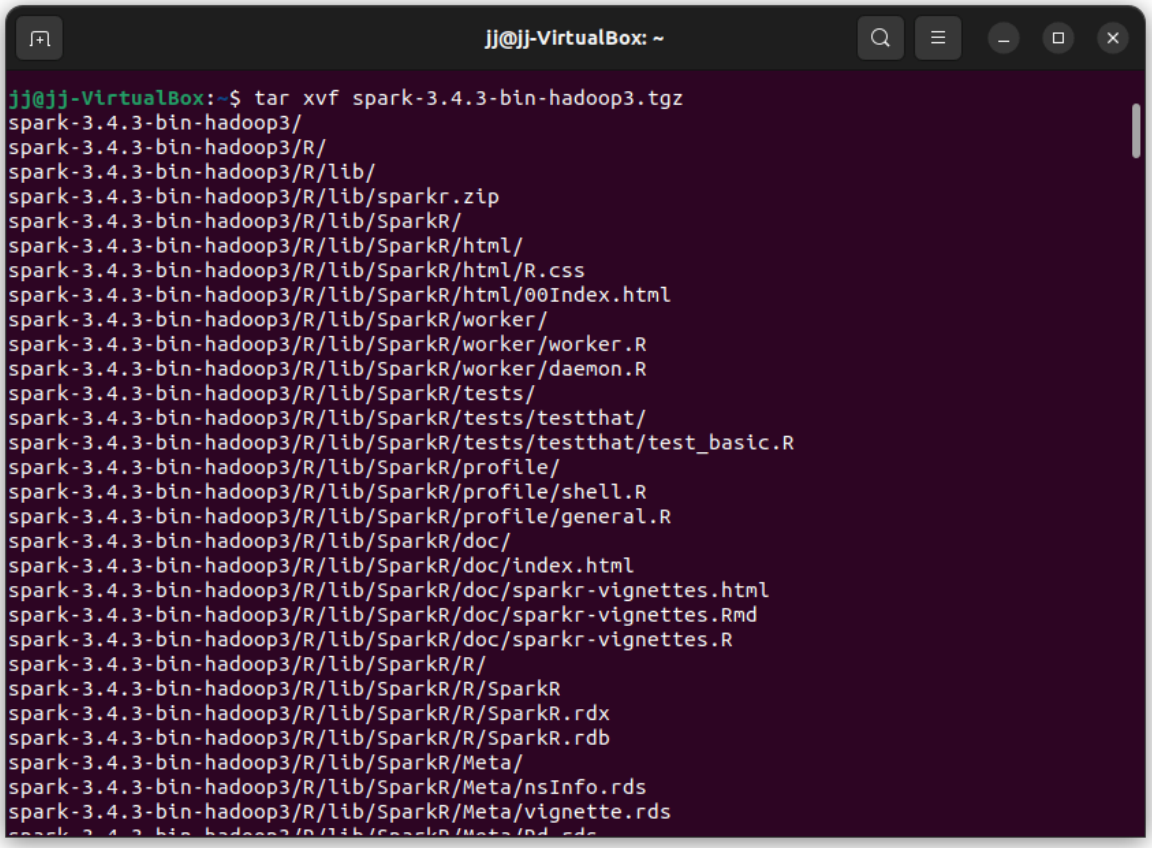
Step 1: Update the System and install Java if not present



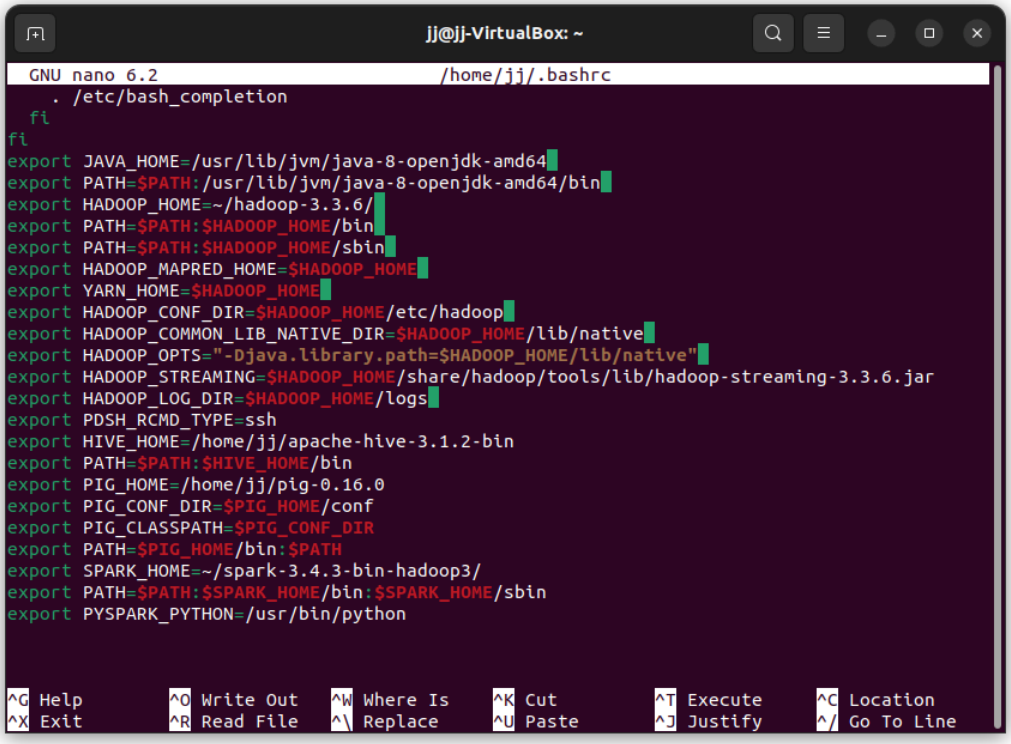
Step 2: Download Apache Spark



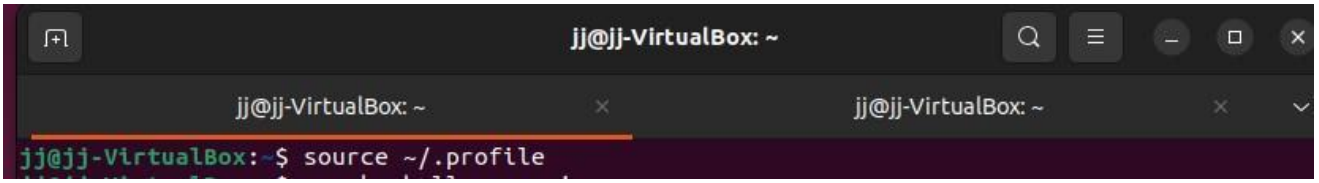
Step 4: Extract the Apache Spark File



Step 5: Set Environment Variables on Ubuntu using nano editor



Step 6: Load the file



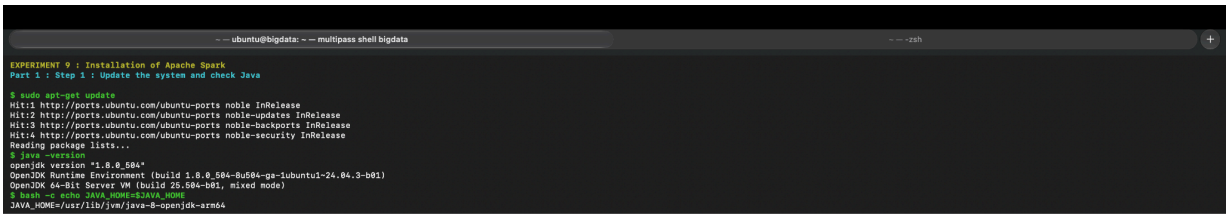
Output:

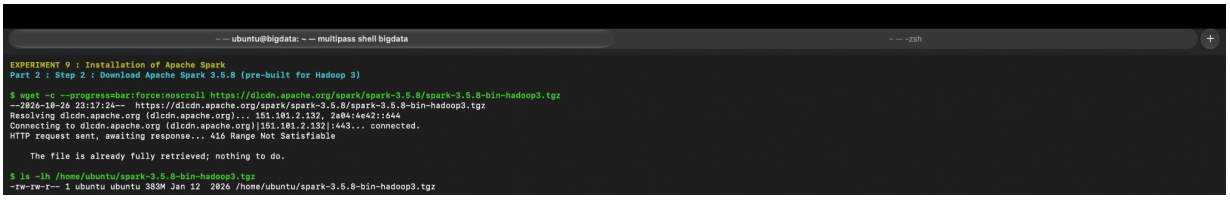
```
jj@jj-VirtualBox:~$ spark-shell
24/04/23 10:22:45 WARN Utils: Your hostname, jj-VirtualBox resolves to a loopback address:
127.0.1.1; using 10.0.2.15 instead (on interface enp0s3)
24/04/23 10:22:45 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel)
.
24/04/23 10:22:51 WARN NativeCodeLoader: Unable to load native-hadoop library for your plat
form... using builtin-java classes where applicable
Spark context Web UI available at http://10.0.2.15:4040
Spark context available as 'sc' (master = local[*], app id = local-1713847972289).
Spark session available as 'spark'.
Welcome to
      ____              __
     / __/__  ___ _____/ /__
    _\ \/ _ \/ _ `/ __/  '_/
   /___/ .__/\_,_/_/ /_/\_\   version 3.4.3
      /_/

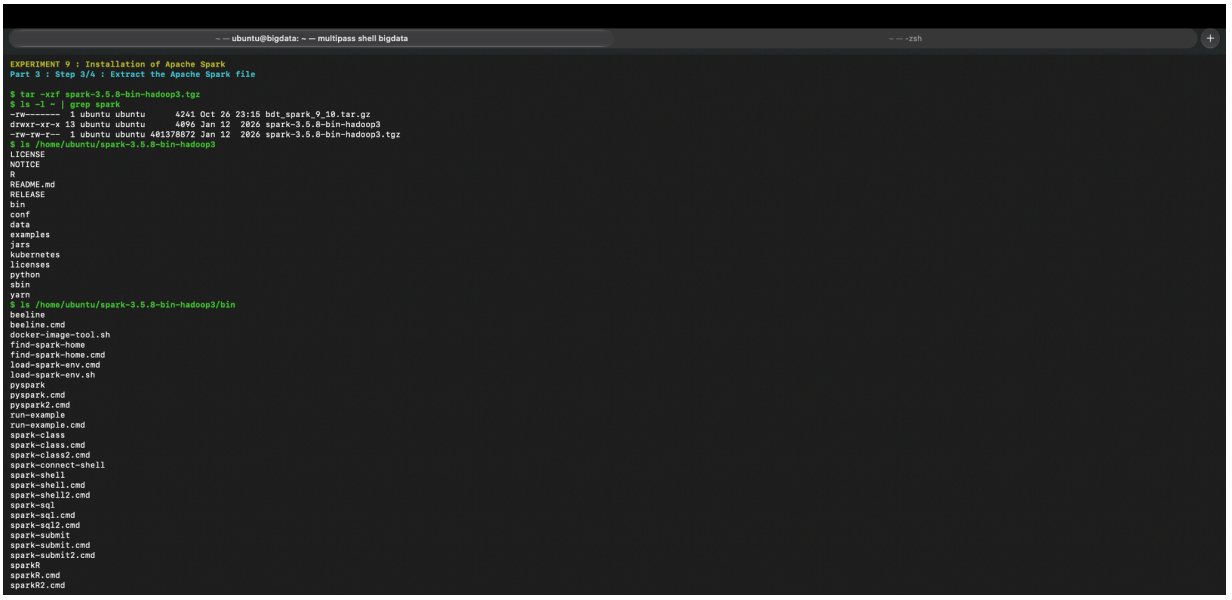
Using Scala version 2.12.17 (OpenJDK 64-Bit Server VM, Java 1.8.0_402)
Type in expressions to have them evaluated.
Type :help for more information.

scala>
```

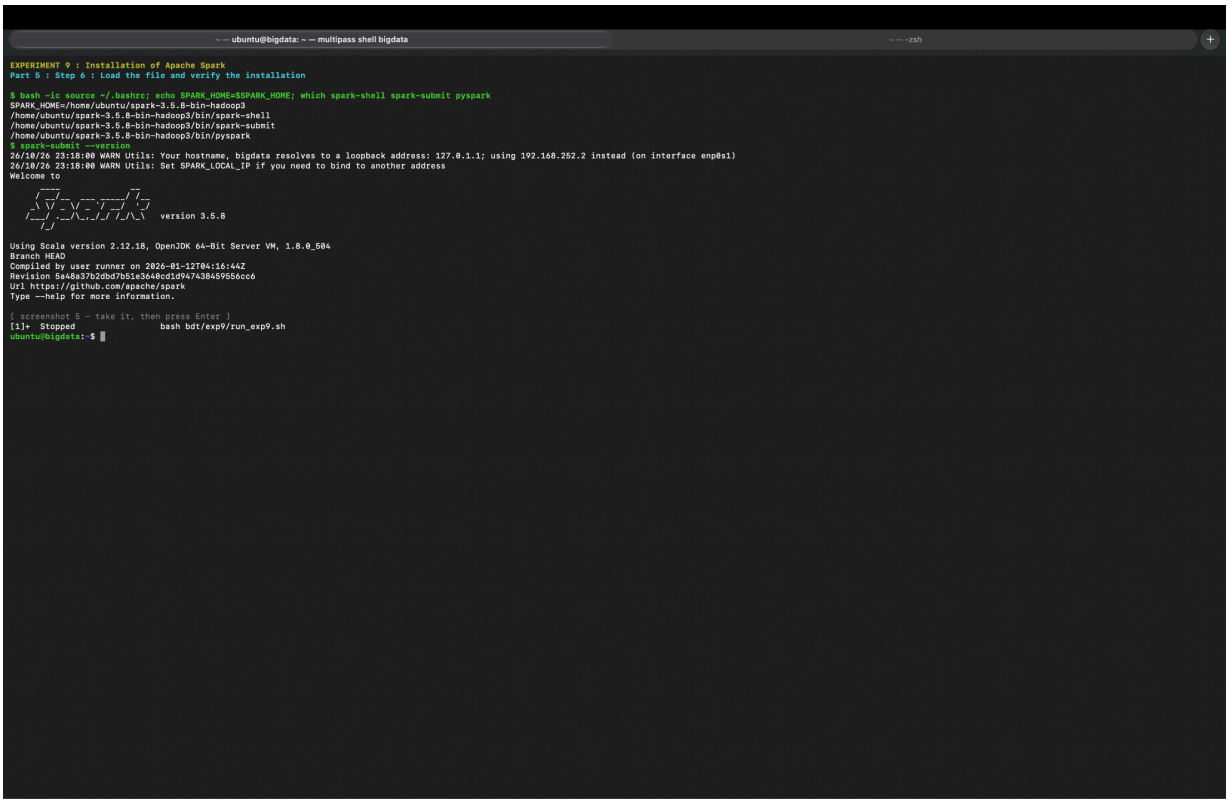
Screenshots :







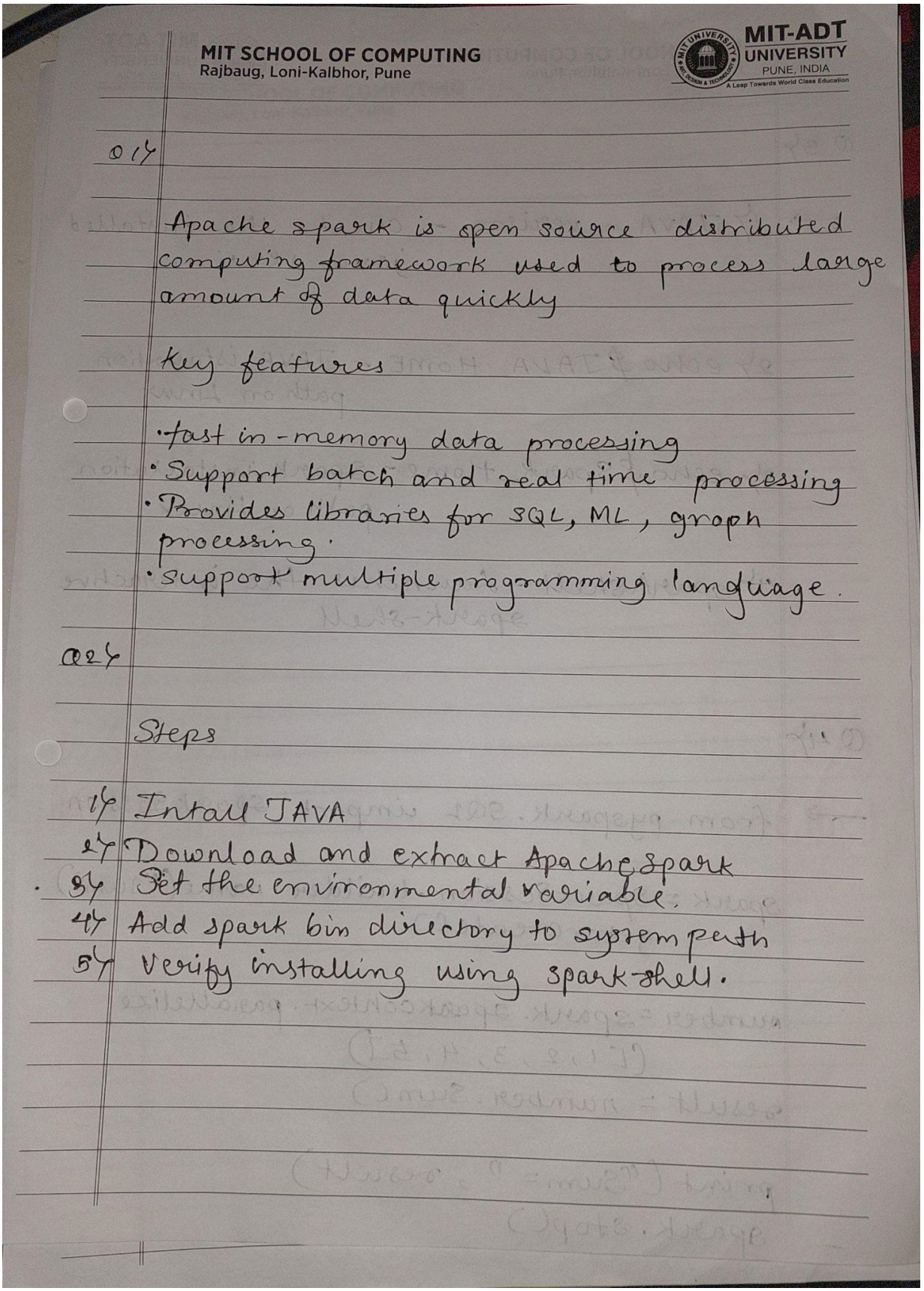




Handwritten Questions

1. What is Apache Spark? Explain its key features and describe how it differs from the traditional MapReduce processing model.
2. List the prerequisites for installing Apache Spark. Explain the steps required to configure Java, Spark, and the necessary environment variables.
3. Write the commands to verify the Java and Spark installations and launch the Spark Shell. Explain the purpose of each command.

4. Write a basic Spark application to create a collection of numbers, calculate their sum, and display the result. Explain how the application can be executed and verified.

5. 

MIT SCHOOL OF COMPUTING
Rajbaug, Loni-Kalbhor, Pune

MIT-ADT UNIVERSITY
PUNE, INDIA
A Leap Towards World Class Education

Q.3)

→ 1) JAVA -version - checks the installed java version.

2) echo $JAVA_HOME - JAVA istallation path on Linux

3) echo $Spark_Home - Spark installation path on Linux

4) spark-shell - Launches the interactive spark-shell

Q.4)

→

```
from pyspark.SQL import SparkSession

spark = SparkSession.builder.name("Sum").
        getorcreate()

number = spark.sparkcontext.parallelize
         ([1, 2, 3, 4, 5])
result = number.Sum()

print("Sum=", result)
spark.stop()
```

Acceptance Criteria

1 The student identifies the required software prerequisites and successfully installs or configures Apache Spark in the laboratory environment.

2	The student correctly configures the required environment variables and verifies the Java and Spark installations.
3	The student successfully launches the appropriate Spark Shell or execution environment without configuration errors.
4	The student executes a basic Spark application, obtains the expected output, and explains the execution results.

Conclusion: Apache Spark 3.5.8 (pre-built for Hadoop 3) was installed on Ubuntu with Java 8. SPARK_HOME and PATH were set in ~/.bashrc, and the installation was verified with spark-submit --version (Spark 3.5.8, Scala 2.12.18). A basic Spark application in the Spark Shell created a collection of numbers 1–10 and calculated their sum as 55. This shows that Spark was installed and configured correctly and can run applications.